

Low-Latency Browser Voice Assistant — State of the Art, September 2026

NOTE

Research for a 3-day take-home. Every latency and pricing claim below carries a source

NOTE

URL; where sources disagree I say so rather than picking a number.

CONTENTS

Summary — read this page only

- 1 Realtime speech-to-speech APIs
- 2 Cascaded components with generous free tiers
- 3 TTS — sub-300 ms time-to-first-audio
- 4 Orchestration frameworks
- 5 Latency budget
- 6 Deployment targets
- 7 Memory across sessions

Recommended build

Summary — read this page only

Build this. Pipecat (open source, Python) + `SmallWebRTCTransport` in the browser → Groq `whisper-large-v3-turbo` STT → Cerebras `gpt-oss-120b` LLM → Cartesia Sonic TTS, with Pipecat's local **Smart Turn v3** turn detector (12 ms CPU inference) doing endpointing. Deploy the agent on **Fly.io** or **Modal** (never Vercel — see below), frontend on Vercel. Memory = structured extraction into Supabase Postgres, written asynchronously after each turn.

Why: every component is free or near-free, Pipecat gives you per-stage TTFB metrics out of the box (`enable_metrics=True`) which is the single most demo-impressive thing you can put on screen, and the cascade is what the assignment asked for.

The three rankings:

GOAL	PICK
(a) Lowest latency	OpenAI <code>gpt-realtime</code> over WebRTC (~300 ms voice-to-voice, single hop). A well-tuned cascade can match it, but only if you fix endpointing — use Deepgram Flux (integrated end-of-turn) or Smart Turn v3, not a fixed VAD silence threshold.
(b) Fastest to build in 3 days	Pipecat + <code>SmallWebRTCTransport</code> , or LiveKit Agents on the LiveKit Cloud free tier (1,000 agent minutes/mo). Both are ~100 lines to a working voice loop.
(c) Most impressive demo	A live latency HUD driven by Pipecat's <code>TTFBMetricsData</code> / OTel turn spans, showing VAD → STT → LLM TTFT → TTS TTFB per turn; plus barge-in, plus a toggle that swaps the cascade for Gemini Live speech-to-speech in the same UI, plus cross-session memory recall ("last time you said you're allergic to shellfish").

The five facts that should shape your architecture:

- 1. Endpointing, not inference, is your latency budget.** A naive 500 ms VAD silence threshold is 500 ms of pure dead air added to every turn — larger than your entire STT + LLM + TTS budget on a fast stack. Semantic turn detection adds ~100 ms of compute but lets you cut the silence wait. ([LiveKit](#), [Deepgram](#))
- 2. WebRTC, not WebSocket, for the browser leg.** WebRTC hits 150–250 ms first-token in browsers; WebSocket-over-TCP adds 100 ms+ spikes on lossy Wi-Fi from head-of-line blocking. OpenAI explicitly recommends WebRTC for browser clients and WebSocket only for server-to-server. ([OpenAI](#), [callsphere](#))
- 3. Vercel cannot host your voice agent.** Native WebSocket support shipped in public beta on 22 June 2026, but it runs inside a Vercel Function and inherits the 300 s max duration — the socket dies mid-conversation. ([Ably](#))
- 4. Gemini Live is ~10× cheaper than OpenAI Realtime on audio** (\$3/M vs \$32/M audio in; \$12/M vs \$64/M audio out) and has a free tier. OpenAI has WebRTC; Gemini Live is WebSocket-only. ([Google](#), [OpenAI](#))

5. **Vendor TTS latency numbers are not comparable to each other.** Coval redefined TTFA on 3 June 2026 to include leading silence inside the stream; one vendor's number went from 172 ms to 430 ms on the methodology change alone. Use the independent Coval leaderboard, not vendor pages. ([Coval methodology](#))

1 Realtime speech-to-speech APIs

OpenAI Realtime API

Model IDs currently listed on the pricing page: `gpt-realtime-2.1`, `gpt-realtime-2.1-mini`, `gpt-realtime-2`, `gpt-realtime-1.5`, `gpt-realtime`, `gpt-realtime-mini`. The `gpt-realtime` model card gives snapshot `gpt-realtime-2025-08-28`, a **32,000 token context window**, 4,096 max output tokens, input modalities text/audio/image, output text/audio, and connectivity over **"WebRTC, WebSocket, or SIP"** at the `v1/realtime` endpoint. ([model card](#), retrieved 18 Sep 2026)

Pricing per 1M tokens ([OpenAI pricing](#), retrieved 18 Sep 2026):

MODEL	AUDIO IN	AUDIO CACHED	AUDIO OUT	TEXT IN	TEXT OUT
<code>gpt-realtime-2.1</code>	\$32	\$0.40	\$64	\$4	\$24
<code>gpt-realtime-2.1-mini</code>	\$10	\$0.30	\$20	\$0.60	\$2.40
<code>gpt-realtime</code> / <code>-2</code> / <code>-1.5</code>	\$32	\$0.40	\$64	\$4	\$16–24
<code>gpt-realtime-mini</code>	\$10	\$0.30	\$20	\$0.60	\$2.40

Real-world per-minute cost: roughly **\$0.05/min** on `gpt-realtime-2.1` base math and **~\$0.016/min** on mini ([Layer3Labs](#)), but a study of 4,000 measured production sessions puts `gpt-realtime-mini` at **\$0.063–\$0.146/min** in practice, because every turn resubmits the full context and input tokens grow linearly with turn count unless you prune ([HackerNoon](#)). With a 32k context window and no pruning you will hit the ceiling on a long demo call.

Browser flow: your server mints an ephemeral key from `POST /v1/realtime/client_secrets`; the browser does SDP exchange against `https://api.openai.com/v1/realtime/calls` with that key. Never ship the real API key. ([OpenAI realtime guide](#))

Related audio model pricing (same page): transcription `gpt-realtime-whisper` \$0.017/min, `gpt-transcribe` \$0.0045/min, `gpt-4o-transcribe` \$0.006/min, `gpt-4o-mini-transcribe` \$0.003/min, `whisper` \$0.006/min. TTS `tts-1` \$15.00/1M chars, `tts-1-hd` \$30.00/1M chars.

Google Gemini Live API

Model IDs ([Gemini pricing](#) and [Live API capabilities](#), retrieved 18 Sep 2026):

- `gemini-3.8-live` — recommended default for low-latency voice dialogue
- `gemini-3.8-live-extended-thinking` — background reasoning + async tool work
- `gemini-3.1-flash-live-preview` — legacy
- `gemini-3.5-live-translate-preview`, `gemini-3.5-transcribe-live`
- `gemini-live-2.5-flash-native-audio` — launched 12 Dec 2025, **scheduled for discontinuation 13 Dec 2026**; do not build on it ([release dates](#))

Both 3.8 Live models released **15 September 2026** — three days before this research.

Pricing per 1M tokens for `gemini-3.8-live`: text in \$0.75, audio in **\$3.00** (or \$0.005/min), text out (including thinking tokens) \$4.50, audio out **\$12.00** (or \$0.018/min). `gemini-3.5-transcribe-live`: audio in \$3.50/M (\$0.005/min), text out \$21.00/M (\$0.004/min).

Free tier is available on `gemini-3.8-live` per the pricing page. Google no longer publishes static per-model RPM/TPM/RPD numbers — the [rate limits doc](#) defers to the AI Studio dashboard at `aistudio.google.com/rate-limit`. Third-party figures for Gemini 3.8 Flash free tier range from ~20 RPD ([pecollective](#)) to ~15 RPM / 1,500 RPD for the general Flash free tier ([Standard Compute, verified 17 Jul 2026](#)). **Check AI Studio before you depend on it** — this is the single least-trustworthy number in this report.

Hard constraints that matter for a demo:

- **Audio-only sessions cap at 15 minutes** (audio+video at 2 minutes). Session resumption / context compression extends this, but you must implement it.

- Context: 128k for native-audio models, 32k for other Live models.
- Turn detection: automatic VAD (default, configurable sensitivity), hybrid VAD (server start-detection + client end-detection), or manual `activityStart` / `activityEnd` .
- **Transport is WebSocket. WebRTC is not offered.** Server-to-server auth by default; browser clients require **ephemeral tokens**.

Speech-to-speech vs cascaded — the actual trade

	SPEECH-TO-SPEECH	CASCADED STT→LLM→TTS
Typical voice-to-voice	400–800 ms; ~300 ms achievable on OpenAI WebRTC	800–1,500 ms typical; 500–700 ms tuned
Prosody / emotion	Preserved end to end (this is the real win)	Lost at STT; TTS re-synthesises flat
Interruption handling	Native, model-level	You build it (VAD + pipeline cancellation)
Model choice	Locked to one vendor's voice model	Swap any STT/LLM/TTS independently
Observability	Black box — one latency number	Per-stage TTFB you can actually optimise
Text transcript	Side-channel, sometimes lossy	First-class, exact
Cost	\$0.016–\$0.146/min (OpenAI); Gemini much cheaper	Free tier feasible end to end
Free tier	Gemini only	Groq + Cerebras + Cartesia = genuinely free

Sources: [Coval voice AI models, 4 May 2026](#); [Reactify 2026 production comparison](#); [futureagi sub-500ms guide](#).

For a take-home, the cascade is the stronger submission — it demonstrates systems thinking, it's what the assignment names, and it lets you show a latency waterfall. Adding an S2S toggle as a second mode is the cheap way to also show you know the alternative.

2 Cascaded components with generous free tiers

Groq

Free-plan rate limits, straight from console.groq.com/docs/rate-limits (retrieved 18 Sep 2026):

MODEL ID	RPM	RPD	TPM	TPD	AUDIO SEC/HR	AUDIO SEC/DA
whisper-large-v3-turbo	20	2,000	—	—	7,200	28,800
whisper-large-v3	20	2,000	—	—	7,200	28,800
openai/gpt-oss-120b	30	1,000	8,000	200,000	—	—
openai/gpt-oss-20b	30	1,000	8,000	200,000	—	—
qwen/qwen3.8-27b	30	1,000	8,000	200,000	—	—
groq/compound	30	250	70,000	—	—	—
groq/compound-mini	30	250	70,000	—	—	—
canopylabs/orpheus-v1-english	10	100	1,200	3,600	—	—
canopylabs/orpheus-arabic-saudi	10	100	1,200	3,600	—	—

⚠ **Two traps in that table.** (1) **8,000 TPM** on the `gpt-oss` models is tight — a voice turn with a 2k-token system prompt and growing history can burn that in a handful of turns. Keep the system prompt short and window the history hard, or put the LLM on Cerebras instead. (2) **Groq's Orpheus TTS free tier is 10 RPM / 100 RPD** — enough for one demo conversation, not enough to develop against. Use Cartesia or Rime for TTS and keep Groq for STT.

STT: `whisper-large-v3-turbo` . Groq's model page states a **216× speed factor** at **\$0.04 per hour** of audio ([GroqDocs](#)). Other write-ups report 217–228×, i.e. an hour of audio in ~15 s ([eesel](#)). The free tier's **28,800 audio seconds/day = 8 hours of audio per day, free** —

far more than a take-home needs. Note this is a *batch* transcription endpoint: you send utterance chunks, you do not get partial hypotheses streaming back the way you do from Deepgram. Budget ~150–300 ms per chunk round-trip and design your endpointing around that.

LLM speeds from [Artificial Analysis](#): gpt-oss-20b (high) **943 t/s**, gpt-oss-20b (low) 919 t/s, Llama 3.1 8B 636 t/s, gpt-oss-120b (high) 472 t/s, Llama 3.3 70B 286 t/s. AA's time-to-first-token figures sit at **0.75–0.85 s**, lowest being gpt-oss-120b (low) at 0.75 s.

⚠ **TTFT numbers conflict badly.** Artificial Analysis reports ~0.8 s TTFT for Groq; other benchmarks claim ~120 ms ([tokenmix](#)). The gap is almost certainly reasoning tokens and prompt length — AA runs long prompts and counts reasoning output before the first *answer* token. **Measure it yourself with your own prompt on day 1.** This is exactly the kind of thing that makes a take-home look rigorous.

TTS: Groq ships Canopy Labs Orpheus ([canopylabs/orpheus-v1-english](#) , [canopylabs/orpheus-arabic-saudi](#)), English + Arabic only, expressive with vocal direction controls, still **Preview** ([GroqDocs](#)). Pricing \$22.00/1M chars English, \$40.00/1M chars Arabic as of 14 Jul 2026; generates at ~100 chars/sec, with one source reporting 95 ms P50 latency ([eesel](#)). PlayAI Dialog v1.0 is \$50/1M chars.

Cerebras

Model catalogue ([inference-docs.cerebras.ai](#), retrieved 18 Sep 2026):

MODEL ID	SPEED	CONTEXT (FREE / PAID)	FREE TIER
gpt-oss-120b	~3,000 tokens/s	65k / 131k	Yes
qwen-3.8-27b	~1,850 tokens/s	64k / 128k	Yes

Free tier: **1,000,000 tokens/day, no credit card**, resetting daily (not a one-time credit) ([Get AI Perks](#), [Adam Holter](#)). Reported per-minute caps: 5 RPM, 30,000 uncached TPM / 90,000 total TPM.

⚠ Third-party sources claim the free tier caps context at 8,192 tokens, while the official model catalogue says 65k free for [gpt-oss-120b](#) . **Verify on signup.** 5 RPM is

also low — fine for a single demo conversation, painful if two people test at once.

Cerebras at ~3,000 t/s is the fastest token generation available to you for free, and at voice-length responses (60–120 tokens) generation time is effectively zero — your latency is entirely TTFT plus network. That makes it the right LLM for the cascade, with Groq `gpt-oss-20b` as the fallback when you hit the 5 RPM wall.

SambaNova

Free tier is the tightest of the three: reported 20 RPM and ~200K tokens/day ([costbench](#)), while the Developer Tier allows **20M tokens/day across all models** ([SambaNova blog](#)). Model line-up runs on SambaNova RDU chips: Llama 3.3 70B, DeepSeek V3.1 and V3.2, MiniMax M2.7 and M3, Gemma 4 31B, GPT-OSS 120B. Published throughput: Llama 3.1 70B at **461 t/s**, 405B at 132 t/s at full precision ([BusinessWire](#)).

Verdict: skip it for a 3-day build. Slower than Cerebras, tighter free tier than Groq, and the extra integration buys you nothing. Mention it in your write-up as evaluated-and-rejected — that reads better than not having looked.

Google Gemini (as the cascade LLM)

`gemini-3.8-flash` is GA, free tier available, 1,048,576 input / 65,536 output tokens, \$0.75/M in and \$3.75/M out on paid ([pecollective](#), [Gemini pricing](#)). Free-tier RPD numbers are unreliable (see §1). Gemini Flash is a reasonable cascade LLM but it is *not* faster than Cerebras or Groq, so use it only if you want Google-native tool calling or you're already using Gemini Live for the S2S comparison mode.

3 TTS — sub-300 ms time-to-first-audio

The independent benchmark

[Coval](#) runs continuous measurement across 26–28 providers, 480 samples/model, English-only. Mirror with the full table at [openbenchmarks.com](#) (rolling 7-day aggregate,

last synced 17 September 2026):

RANK	MODEL	PROVIDER	TTFA MEDIAN	TTFA P95	WER
1	vui	Fluxions	50 ms	96 ms	5.6%
2	qwen3-tts-fast	Nari	63 ms	104 ms	4.2%
3	inworld-tts-2-flash	Inworld AI	69 ms	107 ms	5.8%
4	qwen3-tts-1.7b	Baseten	100 ms	134 ms	6.1%
5	palabra-tts-v1	Palabra	103 ms	145 ms	6.0%
28	gpt-4o-mini-tts	OpenAI	560 ms	4,341 ms	5.1%

An earlier Coval capture (4 May 2026) covers the names you asked about, with IQR — which matters more than the median:

MODEL	TTFA P50	IQR
Gradium TTS	155 ms	2 ms
Cartesia Sonic-3	188 ms	100 ms
ElevenLabs Turbo v2.5	264 ms	28 ms
ElevenLabs Flash v2.5	288 ms	28 ms
Deepgram Aura-2	313 ms	68 ms

([Gradium's write-up of the Coval data](#) — vendor-published but the underlying numbers are Coval's.)

 **Do not mix these with vendor-published numbers.** On 3 June 2026 Coval redefined TTFA as *(first audio chunk arrival – synthesis start) + leading silence inside the stream before the first audible sample*. One provider's figure moved from 171.9 ms to 429.6 ms on the definition change alone, with no model change. Vendor pages quote round-trip-only TTFB: Cartesia Sonic Turbo ~40 ms, ElevenLabs Flash v2.5 ~75 ms, Deepgram Aura-2 ~200 ms streaming, Rime Coda sub-100 ms, PlayAI Play 3.0 Mini ~150 ms, LMNT Aurora ~140 ms, Hume Octave 2 ~150–200 ms ([Coval's own vendor-number roundup](#))). Those are real but measure a different thing. Also note

Coval excludes TLS/WebSocket handshake (~50–200 ms), which you *will* pay unless you hold a warm connection.

Free tiers and credits

PROVIDER	FREE TIER	PAID ENTRY	NOTES
Cartesia	20K credits/mo $\approx$ 27 min , 2 concurrent	Pro \$5/mo $\rightarrow$ 100K credits $\approx$ 133 min, 3 concurrent	cartesia.ai/pricing . Startup \$49/mo (1.25M), Scale \$299/mo (8M). $\sim$ 750 credits/min of audio.
ElevenLabs	10,000 credits/mo $\approx$ 10 min v2 (Flash is 0.5 credits/char $\rightarrow$ $\sim$ 20 min); 2,500 char/generation cap	—	elevenlabs.io/pricing
Deepgram	\$200 free credit	Aura-2 \$0.030/1K chars	deepgram.com/pricing . \$200 is by far the largest free allowance here.
Rime	10K chars/mo, 200+ voices	\$0.03/1K chars (Mist), \$0.05 (Coda)	rime.ai/pricing . Mist v2 $\sim$ 225 ms P50 on Together AI dedicated; sub-150 ms self-hosted.
Groq (Orpheus)	10 RPM / 100 RPD	\$22/1M chars EN	Preview status. Too tight to develop against.
OpenAI	none	tts-1 \$15/1M chars	Slowest in the Coval table.

Recommendation: Cartesia free tier for development, Deepgram's \$200 credit as your safety net. Cartesia Sonic is the best-known name with genuinely competitive latency and the free tier is enough for three days of iteration; if you burn it, Deepgram's \$200 covers Aura-2 TTS *and* Nova-3/Flux STT on the same credit. Its 100 ms IQR is the one weakness — mention it in your write-up as a known tail-latency risk.

4 Orchestration frameworks

The open-source two

Pipecat (Daily) — Python, composable pipeline, you assemble STT/LLM/TTS yourself.

Transports include Daily WebRTC and `SmallWebRTCTransport` (no Daily account needed), plus WebSocket and telephony.

Its killer feature for a take-home is **built-in metrics**

(docs.pipecat.ai/pipecat/fundamentals/metrics):

```
worker = PipelineWorker(
    pipeline,
    params=PipelineParams(
        enable_metrics=True,          # TTFB, TTFA, TTFAT, processing time
        enable_usage_metrics=True,    # LLM tokens, TTS characters
    ),
)
```

That gives you `TTFBMetricsData` (time to first byte per service), `TTFA` (time to first audio, *including* TTS silence padding), `TTFAT` (time to first answer token from the LLM), processing time, and text-aggregation latency. Read them with the built-in `MetricsLogObserver` or your own observer on `MetricsFrame`. There is also OpenTelemetry tracing with Conversation / Turn / Service spans ([SigNoz](#), [Langfuse](#)). **This is your demo HUD, for free.**

Pipecat Cloud (managed): \$0.01–\$0.03/min for active agents (1×/2×/3× tiers), +\$0.005/min SIP, +\$0.018/min PSTN. **Voice WebRTC transport is free for 1:1 human+agent sessions when using a Pipecat Cloud-provisioned Daily key, plus free Krisp minutes and the free Smart Turn model** (daily.co/pricing/pipecat-cloud).

LiveKit Agents — Apache-2 SDK on top of the LiveKit WebRTC media server. Your agent joins a *room* as a participant, so reconnection, barge-in, multi-user, video and avatars come from the RTC layer rather than being bolted on. Native SIP and phone numbers, no Twilio bridge needed.

LiveKit Cloud **Build tier is free**: 5,000 WebRTC participant minutes + **1,000 AI agent minutes/month**, 50 GB transfer, \$2.50 inference credits, one free US local phone number.

Ship is \$50/mo, Scale \$500/mo (livekit.com/pricing).

Third-party end-to-end measurements: LiveKit Agents 750–900 ms, Pipecat-on-Daily 800–950 ms ([forasoft](#)). Treat this as noise — framework overhead is tens of milliseconds; your model stack sets the latency ([dograh](#)).

Choose Pipecat if the take-home is graded on the pipeline (provider swapping, latency instrumentation, showing your work). **Choose LiveKit** if it's graded on the product (robust reconnection, a polished browser client, multi-device). Setup cost for either is realistically **half a day** including a working browser client.

The managed three (closed source)

PLATFORM	PRICE	LATENCY	FREE CREDIT
Vapi	\$0.05/min platform fee + components; realistically \$0.10–0.30/min all-in	—	~\$10 trial, 60+ minutes
Retell	\$0.07+/min flat	~600–620 ms out of the box, lowest among managed	trial credit
Bland	\$0.07–0.12/min bundled	—	trial

Sources: [Retell's own comparison](#), [Automation Labs true-cost breakdown](#), [Layer3Labs on Vapi](#).

None of these is right for a take-home. They hide exactly the engineering the assignment is testing. Retell's ~600 ms is a useful *target line* to beat or match — cite it as your benchmark.

VAD and turn detection

This is where you win or lose the latency game.

COMPONENT	WHAT IT IS	COST / LATENCY	LICENSE
Silero VAD	Acoustic speech/no-speech detector, the	<1 ms per 30 ms chunk on one CPU thread; ONNX build	MIT, github

COMPONENT	WHAT IT IS	COST / LATENCY	LICENSE	
	de-facto default	runs 4–5× faster; ~2 MB JIT model; runs in-browser via WASM		
Pipecat Smart Turn v3	Semantic end-of-turn, Whisper-tiny base + linear classifier, ~8M params, 23 languages	12 ms CPU inference on a modern CPU, 60 ms on a cheap AWS instance; no GPU	BSD-2 , announcement , HF	
LiveKit turn-detector	Fine-tuned Qwen2.5-0.5B-Instruct predicting `<\`	im_end\`	>`, per-language thresholds	Requires Silero VAD with <code>min_silence_duration</code> ≥ 250 ms; semantic layer adds ~100 ms
Deepgram Flux	STT with end-of-turn <i>fused into the ASR model</i> — acoustic + semantic + conversational context	Median EOT <300 ms (~260 ms), p95 1.5 s; saves 200–600 ms vs STT+VAD pipelines; \$0.0065/min EN, \$0.0078/min multilingual	Deepgram , Coval STT review Jun 2026	
Krisp VIVA	Voice isolation — strips background noise <i>and secondary voices</i> before STT; Turn Prediction v3 (~9M params, 30 MB) and Interruption Prediction v1	CPU-only, no GPU; ~30% WER drop in noisy calls	Commercial; free minutes included on Pipecat Cloud , docs	

Practical stack: Silero VAD (cheap gate) + Smart Turn v3 (semantic commit) locally in your Pipecat process. Both free, both CPU, combined cost ~13 ms. If your STT budget allows the swap, Deepgram Flux collapses STT and endpointing into one call and is the single biggest

latency win available — but it costs money and it's Deepgram-only, which undercuts the "provider swappable" story.

5 Latency budget

What "good" means in 2026

BAR	VOICE-TO-VOICE
Feels genuinely human	~300 ms
Excellent	<500 ms
Good / shippable	600–800 ms p50
Typical stitched multi-vendor stack	600–1,700 ms
Unacceptable	>1,200 ms p50

Measured production reality: across a fleet of deployments, **median end-to-end response latency was 680 ms p50 and 1,180 ms p95** ([DestiLabs 2026 benchmark, 10+ projects](#)). Managed platforms measured 600 ms (Retell, fastest) to 950–1,450 ms for typical configurations ([Telnyx](#)). Standard STT-LLM-TTS pipelines land at 800–1,500 ms; speech-to-speech models cut this to 400–800 ms ([Rayvoc](#)).

The sub-500 ms budget

From [futureagi's sub-500ms guide \(2026\)](#):

STAGE	BUDGET
Network (WebRTC transport)	30–50 ms
STT first partial	100–150 ms
LLM TTFT (with prefix caching)	200–300 ms

STAGE	BUDGET
TTS first audio (streaming provider)	80–150 ms
Orchestration + inline guardrail	50–100 ms

An aggressive variant ([Prodinit](#)): streaming STT 60–100 ms (Deepgram Nova-3), LLM first token 100–180 ms, TTS first chunk 40–80 ms (Cartesia Sonic / ElevenLabs Flash), WebRTC transport 20–40 ms.

Note what is missing from both budgets: endpointing. Neither counts the silence you wait through before deciding the user stopped talking. If you run Silero VAD with its default 550 ms `min_silence_duration`, that 550 ms sits *on top* of everything above — more than the entire rest of the budget. This is the number to attack first, and the number to put on your demo HUD.

Your realistic target for this build

With Groq Whisper turbo (batch, ~150–250 ms), Cerebras `gpt-oss-120b` (TTFT-dominated, generation $\approx$ free at 3,000 t/s), Cartesia Sonic (~190 ms TTFA p50), Smart Turn v3 (12 ms

- a ~250 ms trailing silence), and WebRTC (~40 ms):

~700–900 ms voice-to-voice p50 is an honest target. Claiming sub-500 ms on this stack would require Deepgram Flux and a co-located deployment. Being precise and honest about your measured number, with a waterfall showing where it goes, beats an unverified claim.

6 Deployment targets

TARGET	WEBSOCKET / WEBRTC	FREE?	VERDICT FOR A VOICE AGENT
Vercel	Native WS in public beta since 22 Jun 2026 (RFC 6455 upgrade in Vercel Functions; ws, Socket.IO,	Hobby free	 Will not work. The socket lives inside a Function and dies at the 300 s max duration ; clients

TARGET	WEBSOCKET / WEBRTC	FREE?	VERDICT FOR A VOICE AGENT
	FastAPI, Express, Hono, Nitro, Nuxt supported)		must reconnect with backoff. Use Vercel for the Next.js frontend and the ephemeral-token minting route only. (Aby , Vercel community)
Fly.io	Excellent — connection-aware load balancing, global routing, built for long-lived connections	✗ Free allowance removed for orgs created after Oct 2024 ; short trial then card-from-dollar-one	✓ Best fit. Cheap (a few dollars for a take-home), Docker-native, <code>fly deploy</code> from a Dockerfile, regions you can pick to sit near your model providers. (budgetforge)
Railway	Fine	✗ Free tier killed Aug 2023; \$5 one-time trial credit; prepaid credits removed early 2026; Hobby \$5/mo <i>including</i> \$5 usage	✓ Easiest DX of the three. \$5 covers a take-home comfortably.
Render	Fine	✓ Only one with a permanent free tier	⚠ Free services spin down after 15 min idle and take ~1 min to wake . Fatal for "reviewer clicks the link and talks to it" unless you keep it warm with a cron ping.
Cloudflare Workers + Durable Objects	<code>@cloudflare/voice</code> on the Agents SDK: audio over the existing WebSocket into a continuous STT session, conversation history auto-persisted in the DO's SQLite , unified voice+text on one agent instance. RealtimeKit provides TURN + serverless SFU; raw WebRTC audio into Workers as PCM.	Generous free tier	⚠ Architecturally gorgeous, but <code>@cloudflare/voice</code> is experimental/beta and moving fast. Durable Objects solve session persistence <i>and</i> memory in one primitive. High risk, high reward for 3 days. (Cloudflare docs , blog)

TARGET	WEBSOCKET / WEBRTC	FREE?	VERDICT FOR A VOICE AGENT
Modal	Out-of-the-box token streaming, WebRTC and WebSocket support; QuiLLMan voice-chat example ships with a React frontend	✅ \$30/month free compute credits , scale-to-zero, billed per second	✅ Strong second choice , and the only free option with real compute. Python-native (matches Pipecat). Cold starts are the thing to watch. (Modal , QuiLLMan)

Recommendation: Fly.io (~\$3–5 for the project) or Modal (\$30/mo free) for the agent process; Vercel for the Next.js frontend. Put the agent in a region close to your providers — `iad` if you're on Groq/Cerebras US endpoints.

7 Memory across sessions

OPTION	STRENGTH	LONGMEMEVAL	3-DAY VERDICT
mem0	Fastest integration — ~5 lines of Python; hosted free tier <i>or</i> self-host; hybrid vector + graph + KV; ~55k GitHub stars (May 2026)	~49%	✅ If you want a named library on your CV line
Zep / Graphiti	Bi-temporal knowledge graph, async summarisation + entity extraction	71.2%	⚠️ Best recall by a wide margin, but a whole server to run
Letta	Agent-as-its-own-memory runtime, self-editing memory, sleep-time compute	—	❌ Biggest architectural commitment; rewrites your agent
Structured extraction → Postgres/Supabase/SQLite	Zero new infra, total control, trivially explainable in a write-up	—	✅ Do this

Sources: [Medium 6-dimension comparison](#), [Developers Digest 2026](#), [particula.tech](#).

The 3-day pattern

1. **Read path (session start, off the critical path):** load the user's fact rows into the system prompt as a short bulleted block. Cap it — 15–20 facts, ~300 tokens. On Groq's 8k TPM free tier this matters.
2. **Write path (after each turn, `asyncio.create_task` , never awaited in the turn):**
one cheap LLM call with a strict JSON tool schema — `{facts: [{key, value, confidence}]}` — over the last exchange. Upsert by `key` into `user_facts(user_id, key, value, updated_at)` .
3. **Storage:** Supabase Postgres (free tier, hosted, gives you auth for free) or plain SQLite if you're on Fly with a volume. If you go Cloudflare Durable Objects, the DO's built-in SQLite *is* your memory store and persists across reconnections and deploys automatically.

⚠ **Never put memory writes in the turn's critical path**, and never let memory grow the prompt unboundedly — as context grows, TTFT grows, and TTFT is your largest controllable latency component.

Demo moment worth engineering: end session, hard-refresh, start a new one, and the agent opens with something only the previous session could have told it. That one interaction is worth more than any latency number.

Recommended build

```
Browser (Next.js on Vercel)
└─ WebRTC (SmallWebRTCTransport) → Pipecat agent on Fly.io / Modal
                                   │
                                   │ Silero VAD (<1ms) + Smart Turn v3 (12ms CPU, BSD-2)
                                   │
```

STT	Groq whisper-large-v3-turbo	(216×, free: 8 hrs audio/day)
LLM	Cerebras gpt-oss-120b	(~3000 t/s, free: 1M tokens/day)
	└ fallback Groq gpt-oss-20b	(943 t/s, free: 1k RPD)
TTS	Cartesia Sonic	(~188ms TTFA p50, free: 20k credits)
	└ fallback Deepgram Aura-2	(\$200 free credit)
Memory	async structured extraction	→ Supabase Postgres
Metrics	Pipecat enable_metrics=True	→ live latency HUD in the browser

Day plan. Day 1: Pipecat voice loop working locally over `SmallWebRTCTransport`, all four providers wired, metrics printing to console. *Measure your own TTFT numbers* — the published ones disagree by 6×. Day 2: browser UI, latency HUD, barge-in, memory read/write, deploy to Fly. Day 3: tune endpointing (this is where your remaining latency is), add the Gemini Live speech-to-speech toggle as a second mode, write up the measured latency waterfall.

What to put in the write-up that most candidates won't: a p50/p95 table of your own measured per-stage latencies, the endpointing trade-off you chose and why, and the Artificial-Analysis-vs-vendor TTFT discrepancy you found and resolved by measuring.